

Django

2026

Student Guide

Django 5.2 LTS • Django REST Framework 3.15 • Python 3.12+

■ ■ What We Will Build

Across these 7 days we will build a **Task Manager** — a real web application that lets users create, track, and manage their tasks. Think of it like a personal to-do list where each task has a title, description, status (pending / in progress / done), priority (low / medium / high), and a due date.

Why a Task Manager?

A Task Manager has natural business logic that makes both Django and REST APIs feel real. Tasks belong to users, have statuses, priorities, and deadlines — so you'll need models, forms, authentication, and API permissions. It's complex enough to be meaningful, simple enough to build in a week.

Days 1–4 build the web application — a browser-based interface where you can log in, create tasks, edit them, and delete them.

Days 5–7 build a REST API on top of the same project — so a mobile app or React frontend could also interact with your tasks using JSON instead of HTML.

■ Contents

Day 1	Setup, Models & Django Admin
Day 2	Views, URLs & Templates
Day 3	Forms — Create, Edit & Delete
Day 4	Authentication & Permissions

DJANGO

DJANGO

DJANGO

DJANGO

Day 5 Intro to DRF — Serializers & API Views

DRF

Day 6 APIView & Token Authentication

DRF

Day 7 ViewSets, Routers & Filtering

DRF

Day 1 — Setup, Models & Django Admin

Install Django, understand the structure, build the Task model, explore the admin panel

■ How the Web Works

Every website follows the same cycle. Your browser sends a **request** to a server. The server processes it and sends back a **response** — usually an HTML page. Django is the program that runs on the server. It receives requests, does the logic, and returns responses.

■ *Analogy: Django is a restaurant kitchen. The customer (browser) places an order (request). The kitchen (Django) prepares the meal and sends it out (response). Customers never see the kitchen — they just get the result.*

■ pip — Python's Package Manager

pip is a command-line tool that comes with Python. It lets you download and install third-party libraries from the internet. Django itself is just a pip package — someone wrote it, published it, and you install it for free.

```
pip install django==5.2 # Install Django version 5.2
pip install django # Install the latest version (not recommended for learning)
pip list # See everything currently installed
pip freeze > requirements.txt # Save a list of all packages to a file
```

■ requirements.txt is how you share your project. Anyone can run 'pip install -r requirements.txt' to get the exact same packages you have.

■ Virtual Environment — Why You Need One

Imagine you have two projects. Project A needs Django 4.2, Project B needs Django 5.2. If you install packages globally on your computer, they will conflict. A **virtual environment** solves this by creating a private, isolated copy of Python for each project — packages in one project never affect another.

```
# Create the virtual environment (run once per project)
python -m venv venv

# Activate it (run this every time you open a new terminal)
source venv/bin/activate # Mac / Linux
venv\Scripts\activate # Windows

# You will see (venv) appear at the start of your terminal line
(venv) $ ← this means you are inside the virtual environment

# Deactivate when you are done
deactivate
```

■ Always activate your virtual environment before running any Django commands. If you forget, packages install globally and things break in hard-to-debug ways.

■ Installing Django & Creating Your Project

We use **Django 5.2 LTS**. LTS means Long-Term Support — this version receives security updates for 3 years, making it the safest choice for learning and production.

```
pip install django==5.2
python -m django --version # Should print: 5.2

django-admin startproject taskmanager # Create the project
cd taskmanager # Enter the folder
python manage.py runserver # Start the server

# Open http://127.0.0.1:8000/ — you should see the Django welcome page
```

■ Project Structure — What Each File Does

```
taskmanager/ ← outer folder (just a container, you can rename it)
■■■ manage.py ← YOUR COMMAND CENTER (explained below)
■■■ taskmanager/ ← the actual Python package (same name as project)
■■■ settings.py ← ALL configuration: database, installed apps, keys...
■■■ urls.py ← the main URL router for the whole project
■■■ wsgi.py ← how production servers talk to Django (ignore for now)
■■■ asgi.py ← same as wsgi but for async apps (ignore for now)
```

manage.py is the file you will use every single day. Every Django command runs through it:

Command	What it does
python manage.py runserver	Start the development server at http://127.0.0.1:8000/
python manage.py startapp name	Create a new app inside the project
python manage.py makemigrations	Generate migration files when you change a model
python manage.py migrate	Apply pending migrations to the database
python manage.py createsuperuser	Create an admin account to log into /admin/
python manage.py shell	Open an interactive Python shell with Django loaded

■ Django Project vs Django App — A Critical Distinction

This confuses almost every beginner, so read carefully. In Django, a **project** and an **app** are two different things that work together.

Project

App

The entire website. Created once with <code>django-admin startproject</code> . Contains global settings, the root URL file, and ties everything together. There is exactly ONE project per website.	A self-contained module that handles one feature area. A project can have MANY apps. For example: a 'tasks' app, a 'users' app, a 'notifications' app. Apps can even be reused across different projects.
<code>django-admin startproject taskmanager</code>	<code>python manage.py startapp tasks</code>

■ *Analogy: The project is the whole restaurant. Apps are the departments inside it — the kitchen, the bar, the bakery. Each department runs independently but they're all part of the same restaurant.*

Create the tasks app and register it:

```
python manage.py startapp tasks

# Then open taskmanager/settings.py and add 'tasks' to INSTALLED_APPS:
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'tasks', # ← Add your app here. Django won't recognize it otherwise.
]
```

■ If you forget to add your app to `INSTALLED_APPS`, migrations won't run for it and templates inside it won't be found. Always register your app.

■ The Task Model

A **Model** is a Python class that represents a database table. Each attribute you define becomes a column in that table. Django reads your model and creates the table automatically — you never write SQL.

What is a database table? Think of it exactly like a spreadsheet. Column headers are field names. Each row is one task record.

```
# tasks/models.py
from django.db import models
# Note: we will import User and add the owner field in Day 4.

class Task(models.Model):
    # choices= restricts the field to a fixed set of values
    STATUS_CHOICES = [
        ('pending', 'Pending'), # (stored_in_db, human_readable_label)
        ('in_progress', 'In Progress'),
        ('done', 'Done'),
    ]
    PRIORITY_CHOICES = [
        ('low', 'Low'),
        ('medium', 'Medium'),
        ('high', 'High'),
    ]
```

```

title = models.CharField(max_length=200)
# CharField = short text with a character limit. Good for titles and names.

description = models.TextField(blank=True)
# TextField = long text, no limit. blank=True means this field is optional.

status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='pending')
priority = models.CharField(max_length=10, choices=PRIORITY_CHOICES, default='medium')

due_date = models.DateField(null=True, blank=True)
# null=True → the database column can store NULL (no value)
# blank=True → the form field is optional (you can submit without filling it)

created_at = models.DateTimeField(auto_now_add=True)
# auto_now_add=True → Django sets this automatically when the record is CREATED.
# You never set it manually. It never changes after creation.

updated_at = models.DateTimeField(auto_now=True)
# auto_now=True → Django updates this automatically every time the record is SAVED.

class Meta:
# NOTE: The 'owner' ForeignKey field is added in Day 4 when we introduce authentication.
# On Day 1 we keep the model simple and use the admin to create tasks directly.
ordering = ['-created_at']
# class Meta is where you put model-level configuration – things that affect
# the model as a whole, not individual fields.
# ordering = ['-created_at'] means: whenever you query tasks, return them
# newest first by default. The '-' means descending order.

def __str__(self):
return f"{self.title} [{self.get_status_display()}]"
# __str__ controls what text represents this object.
# get_status_display() is auto-generated by Django for fields with choices=.
# It returns the human-readable label: 'In Progress' instead of 'in_progress'.

```

■ Migrations — Syncing Your Code with the Database

Every time you create or change a model, you must run **migrations**. A migration is a file Django generates that describes what changed. Think of it as a version history for your database structure.

```

# Step 1: Generate the migration file (Django looks at your models and records what changed)
python manage.py makemigrations
# Output: Migrations for 'tasks': tasks/migrations/0001_initial.py

# Step 2: Apply the migration to the actual database
python manage.py migrate
# Output: Running migrations... OK

# Rule: Every time you add or change a field in models.py, run BOTH commands.

```

■ If you change a model but skip migrations, your code and database will be out of sync — you'll get confusing errors. Always run makemigrations then migrate after any model change.

■ Django Admin — Free Management Dashboard

Django automatically generates a fully functional admin panel at **/admin/**. From here you can add, edit, view, and delete any data in your database through a clean web interface — with zero extra code from you.

To use it, you need to do two things: register your model and create a superuser account.

```
# tasks/admin.py – register the Task model so it appears in the admin
from django.contrib import admin
from .models import Task

@admin.register(Task)
class TaskAdmin(admin.ModelAdmin):
    list_display = ['title', 'status', 'priority', 'due_date']
    # list_display controls which columns appear in the task list table
    # Note: 'owner' is added here in Day 4 after we add the field.

    list_filter = ['status', 'priority']
    # list_filter adds a sidebar with clickable filters

    search_fields = ['title', 'description']
    # search_fields adds a search box that searches these fields

    # Create a superuser (admin account) – run this in your terminal
    python manage.py createsuperuser
    # Django will prompt you for:
    # Username: (choose any)
    # Email: (optional, press Enter to skip)
    # Password: (you won't see it as you type – that's normal)

    # Then start the server and visit:
    # http://127.0.0.1:8000/admin/
    # Log in with the username and password you just created
```

■ The Django Admin is one of the most powerful features of Django. In real projects it's used by non-technical team members to manage content without writing any code. You get it completely free.

— ■ Day 1 Lab

1. Create the Task model exactly as shown above in tasks/models.py
2. Add 'tasks' to INSTALLED_APPS in settings.py
3. Run makemigrations then migrate
4. Register the Task model in admin.py with list_display, list_filter, and search_fields
5. Create a superuser using python manage.py createsuperuser
6. Log into /admin/ and create 5 tasks with different statuses and priorities

■ Bonus Challenge

Add a `completed_at` field: `completed_at = models.DateTimeField(null=True, blank=True)` This field will stay empty (null) until a task is marked done. You set it manually in the admin for now. Run `makemigrations` and `migrate` again after adding it — notice Django creates a new migration file.

Day 2 — Views, URLs & Templates

Display task data in the browser — the MTV pattern fully connected

■ The MTV Pattern — How Django Thinks

Django organizes every web request through three layers called **MTV**. Understanding this flow is the key to understanding Django.

M — Model	T — Template	V — View
Manages data. Talks to the database. tasks/models.py	Manages presentation. HTML files shown to the user. tasks/templates/	The brain. Receives requests, fetches data, returns responses. tasks/views.py

The full request flow: User visits /tasks/ → Django checks urls.py → finds matching pattern → calls the view function → view fetches data from the model → view passes data to the template → template renders HTML → HTML sent back to browser.

■■ URL Routing

When a request arrives, Django reads **urls.py** from top to bottom looking for a matching pattern. Once it finds one, it calls the corresponding view function.

```
# tasks/urls.py (create this new file inside the tasks folder)
from django.urls import path
from . import views
# The dot in '. import views' means 'import from the current folder (tasks)'

app_name = 'tasks'
# app_name creates a namespace so you can write {% url 'tasks:task_list' %}
# instead of just {% url 'task_list' %} – avoids name clashes between apps

urlpatterns = [
    path('', views.task_list, name='task_list'),
    # '' means the root URL: http://127.0.0.1:8000/
    path('<int:pk>/', views.task_detail, name='task_detail'),
    # <int:pk> is a URL parameter – Django captures the number and passes it
    # to the view as a variable called 'pk' (primary key / ID of the task)
]

# taskmanager/urls.py – connect the tasks app URLs to the main project
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('tasks.urls', namespace='tasks')),
```

```
# include() hands off URL matching to tasks/urls.py
# namespace='tasks' matches the app_name we set above
]
```

■ Views

A **view** is a Python function that receives an HTTP request object and must return an HTTP response. The request object contains everything about what the user asked for.

```
# tasks/views.py
from django.shortcuts import render, get_object_or_404
from .models import Task

def task_list(request):
    tasks = Task.objects.all()
    # Task.objects.all() returns every Task from the database as a QuerySet
    # A QuerySet is a collection of database records – like a list, but smarter.
    # Django doesn't actually hit the database until you use the data (lazy evaluation).

    context = {
        'tasks': tasks,
        'total': tasks.count(),
        'pending': tasks.filter(status='pending').count(),
        'in_progress': tasks.filter(status='in_progress').count(),
        'done': tasks.filter(status='done').count(),
    }
    # context is a plain Python dictionary.
    # Keys become variable names inside the template.
    # 'tasks' → {{ tasks }} in the template. 'total' → {{ total }}. And so on.

    return render(request, 'tasks/task_list.html', context)
# render() does three things:
# 1. Finds the template at tasks/templates/tasks/task_list.html
# 2. Fills in the template tags with data from context
# 3. Returns the resulting HTML as an HTTP response

def task_detail(request, pk):
    # 'pk' comes from the URL: visiting /5/ gives pk=5
    task = get_object_or_404(Task, pk=pk)
    # get_object_or_404 tries to fetch the task with this pk.
    # If found → returns it. If not found → returns a 404 page automatically.
    # Without this you'd get an ugly error if someone visits /999/ and task 999 doesn't exist.
    return render(request, 'tasks/task_detail.html', {'task': task})
```

■ Templates

Create this folder structure inside your tasks app:

```
tasks/
  ■■■ templates/
  ■■■ tasks/ ← The app name folder is required
  ■■■ base.html ← Shared layout (navbar, footer)
  ■■■ task_list.html ← Extends base, shows all tasks
```

■ The double folder (templates/tasks/) prevents name clashes. If two apps both had a 'task_list.html', Django needs the app name to tell them apart. Always use templates/appname/filename.html.

base.html — the shared layout. Every other template will inherit this:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Task Manager</title>
</head>
<body>
<nav>
<a href="{% url 'tasks:task_list' %}">All Tasks</a>
</nav>

{% block content %}
<!-- Child templates replace everything between these tags with their own content -->
{% endblock %}
</body>
</html>
```

task_list.html — extends base and fills in the content block:

```
{% extends 'tasks/base.html' %}
{# This MUST be the very first line – no blank lines before it #}

{% block content %}

<h1>My Tasks ({{ total }} total)</h1>
<p>Pending: {{ pending }} | In Progress: {{ in_progress }} | Done: {{ done }}</p>

{% for task in tasks %}
<div>
<h3>
<a href="{% url 'tasks:task_detail' task.pk %}">{{ task.title }}</a>
{# {% url 'tasks:task_detail' task.pk %} generates: /5/ for a task with pk=5 #}
</h3>
<span>{{ task.get_status_display }}</span>
<span>Priority: {{ task.get_priority_display }}</span>
{% if task.due_date %}
<span>Due: {{ task.due_date }}</span>
{% endif %}
</div>

{% empty %}
{# {% empty %} runs only when 'tasks' is an empty list #}
<p>No tasks yet!</p>
{% endfor %}

{% endblock %}
```

■ {{ task.get_status_display }} calls the auto-generated Django method that returns the human-readable choice label. If status is 'in_progress', get_status_display returns 'In Progress'.

—■ Day 2 Lab

1. Create tasks/urls.py and connect it in taskmanager/urls.py
2. Build the task_list view and render task_list.html with real data from the database
3. Build the task_detail view and a task_detail.html template showing all task fields
4. Create base.html with a navbar and make both templates extend it
5. Add a 'Back to list' link on the detail page using {% url %}

■ Bonus Challenge

Color-code each task row based on priority using inline HTML styles: red border for high, yellow for medium, green for low. Use {% if task.priority == 'high' %} inside the template to check the value.

Day 3 — Forms — Create, Edit & Delete

Let users create, update, and delete tasks through the browser

■ GET vs POST — How Browsers Send Data

HTML forms send data to the server using one of two HTTP methods. Choosing the right one matters for both functionality and security.

GET	POST
Used to retrieve or display data. Data goes in the URL: <code>/search/?q=django</code> Visible in the browser address bar. Bookmarkable.	Used to send or save data. Data goes in the request body — hidden from the URL. Not bookmarkable. Required for creating, editing, deleting.

■ Every POST form must include `{% csrf_token %}` inside the tag. Without it Django returns a 403 Forbidden error. CSRF (Cross-Site Request Forgery) is an attack where a malicious website tricks your browser into submitting a form on another site. The token prevents this by proving the form came from YOUR site.

■ ModelForm — Auto-Generated Forms from Your Model

Writing HTML form fields by hand for every model field is tedious. Django's **ModelForm** reads your model and generates the form fields automatically. It also handles validation (checking that required fields are filled, types are correct, etc.) and can save directly to the database.

```
# tasks/forms.py (create this new file inside the tasks folder)
from django import forms
from .models import Task

class TaskForm(forms.ModelForm):

    class Meta:
        # class Meta is a special inner class where you put configuration
        # for the ModelForm itself – not for individual fields.
        # It tells Django: 'which model are we building a form for?'
        # and 'which fields should appear in the form?'

        model = Task
        # model = the model this form is based on

        fields = ['title', 'description', 'status', 'priority', 'due_date']
        # fields = the list of model fields to include as form fields.
        # 'created_at' and 'updated_at' are excluded – Django sets them automatically.
        # 'owner' will be added in Day 4 once we introduce authentication.

        widgets = {
            'due_date': forms.DateInput(attrs={'type': 'date'}),
```

```
# widgets = lets you customize how individual fields are rendered in HTML.
# type='date' makes the browser show a date picker instead of a plain text box.
'description': forms.Textarea(attrs={'rows': 4}),
}
```

■ Create, Edit & Delete Views

```
# tasks/views.py – add these imports and views
from django.shortcuts import render, get_object_or_404, redirect
from .forms import TaskForm

def create_task(request):
    form = TaskForm(request.POST or None)
    # request.POST or None is a shortcut:
    # If the form was submitted (POST), use request.POST as the data.
    # If the page was just visited (GET), use None → creates an empty form.

    if form.is_valid():
        # is_valid() runs all validation:
        # - Are required fields filled?
        # - Is the data the right type?
        # - Any custom validators you added?
        # Returns True only if everything passes.

        form.save() # Save directly – owner field doesn't exist yet (added in Day 4)
        return redirect('tasks:task_list')
    # redirect() sends the user to another URL.
    # After creating a task, we send them back to the task list.

    return render(request, 'tasks/task_form.html', {'form': form, 'action': 'Create'})

def edit_task(request, pk):
    task = get_object_or_404(Task, pk=pk)
    form = TaskForm(request.POST or None, instance=task)
    # instance=task tells ModelForm: 'pre-fill the form with this task's existing data'.
    # On POST, it updates that same task instead of creating a new one.

    if form.is_valid():
        form.save()
        return redirect('tasks:task_detail', pk=pk)
    return render(request, 'tasks/task_form.html', {'form': form, 'action': 'Edit'})

def delete_task(request, pk):
    task = get_object_or_404(Task, pk=pk)
    if request.method == 'POST':
        # We only delete on POST – not GET.
        # This prevents accidental deletion if someone just visits the URL.
        task.delete()
        return redirect('tasks:task_list')
    # On GET: show a confirmation page asking 'are you sure?'
    return render(request, 'tasks/task_confirm_delete.html', {'task': task})
```

Add these URL patterns to **tasks/urls.py**:

```
path('create/', views.create_task, name='create_task'),
path('<int:pk>/edit/', views.edit_task, name='edit_task'),
```

```
path('<int:pk>/delete/', views.delete_task, name='delete_task'),
```

The shared form template — works for both create and edit:

```
{% extends 'tasks/base.html' %}
{% block content %}
<h2>{{ action }} Task</h2>
<form method="POST">
{% csrf_token %}
{{ form.as_p }}
{# as_p renders each form field wrapped in a <p> tag – quick and readable #}
<button type="submit">{{ action }}</button>
<a href="{% url 'tasks:task_list' %}">Cancel</a>
</form>
{% endblock %}
```

— ■ Day 3 Lab

1. Create tasks/forms.py with TaskForm
2. Build create_task, edit_task, and delete_task views
3. Add the three URL patterns to tasks/urls.py
4. Create task_form.html and task_confirm_delete.html templates
5. Test the full cycle: create a task, edit it, then delete it
6. Add Edit and Delete buttons to the task detail page

■ Bonus Challenge

Add custom validation to TaskForm. Override the clean_due_date method: if the due_date is in the past, raise forms.ValidationError('Due date cannot be in the past.'). Django will display this error automatically next to the field.

Day 4 — Authentication & Permissions

Login, logout, register — and make sure users can only see their own tasks

■ Django's Built-in Authentication System

Django ships with a complete, production-ready authentication system. You get a **User** model, login and logout views, password hashing, and session management — all without writing it yourself.

■ *Analogy: Authentication is like a hotel key card. When you check in (login), the hotel gives your browser a session ID (key card). Every time you visit a page, your browser sends that card so Django knows who you are — without you re-entering your password on every click. Checking out (logout) cancels the card.*

■ What is a Session?

HTTP is stateless — each request is independent and the server doesn't remember the previous one. **Sessions** solve this. When you log in, Django stores a session ID in your browser as a cookie. On every subsequent request, your browser sends that cookie back, and Django knows who you are. You don't manage sessions manually — Django handles all of it.

■ First: Add the owner Field to the Task Model

Now that we are introducing users, we need to link each task to the user who created it. We do this by adding an **owner** ForeignKey field to the Task model. This is the field we deliberately left out on Day 1.

```
# tasks/models.py — add these two lines
from django.contrib.auth.models import User # import the built-in User model

class Task(models.Model):
    ... # all existing fields stay the same
    owner = models.ForeignKey(User, on_delete=models.CASCADE, related_name='tasks')
    # ForeignKey = a link to another model. Each task belongs to exactly one User.
    # on_delete=CASCADE → if the user is deleted, all their tasks are deleted too.
    # related_name='tasks' → lets you write user.tasks.all() to get a user's tasks.

# Run migrations to add the new column to the database
python manage.py makemigrations
# Django will ask: 'It is impossible to add a non-nullable field owner...'
# Choose option 1 and enter: 1 (to use the superuser with id=1 as default)
python manage.py migrate
```

■ When you add a ForeignKey to an existing table that already has rows, Django needs a default value for existing rows. Choosing id=1 (your superuser) is fine for development. In production you would plan this migration more carefully.

■ Setting Up Login, Logout & Register

Update `taskmanager/urls.py`:

```
from django.contrib import admin
from django.urls import path, include
from django.contrib.auth import views as auth_views
from tasks import views as task_views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('tasks.urls', namespace='tasks')),

    path('login/', auth_views.LoginView.as_view(
        template_name='tasks/login.html'), name='login'),
    # LoginView is Django's built-in view that handles the login form, validation,
    # and session creation. You just need to give it a template to render.
    # as_view() converts the class into a callable view function.
    # This is needed because Django's URL patterns expect a function, not a class.
    # as_view() creates that function for you from the class.

    path('logout/', auth_views.LogoutView.as_view(), name='logout'),
    path('register/', task_views.register, name='register'),
]
```

Add to `settings.py`:

```
LOGIN_URL = '/login/' # Where to redirect guests who try to access protected pages
LOGIN_REDIRECT_URL = '/' # Where to go after a successful login
LOGOUT_REDIRECT_URL = '/login/' # Where to go after logging out
```

👉 ■ Register View

Django provides **UserCreationForm** — a ready-made registration form with username, password, and password confirmation. You don't build it yourself.

```
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth import login

def register(request):
    form = UserCreationForm(request.POST or None)
    if form.is_valid():
        user = form.save() # Creates the new User in the database
        login(request, user) # Logs them in immediately after registering
        return redirect('tasks:task_list')
    return render(request, 'tasks/register.html', {'form': form})
```

■ @login_required — Protecting Views

The **@login_required** decorator protects a view from unauthenticated users. If someone who isn't logged in tries to visit the URL, Django automatically redirects them to the login page. After logging in, Django sends them back to the page they originally requested.

A **decorator** is a Python feature that wraps a function to add behavior. You place it directly above the function with an **@** symbol.

```
from django.contrib.auth.decorators import login_required
```

```

@login_required # Add this above EVERY task view
def task_list(request):
    # Only logged-in users reach this line.
    # Guests are automatically redirected to /login/?next=/
    # After logging in, Django sends them back to /

    tasks = Task.objects.filter(owner=request.user)
    # Now that we have the owner field and @login_required,
    # filter to show only the logged-in user's tasks.
    # Without this filter every user sees every task in the database.
    ...

@login_required
def edit_task(request, pk):
    # IMPORTANT: always filter by owner here too
    task = get_object_or_404(Task, pk=pk, owner=request.user)
    # If task 5 belongs to Alice and Bob tries to visit /5/edit/,
    # get_object_or_404 returns 404 because pk=5 AND owner=Bob finds nothing.
    ...

```

■ Using the User in Templates

The currently logged-in user is always available as `user` in every template — you don't need to pass it via context:

```

{% if user.is_authenticated %}
<p>Welcome, {{ user.username }}!</p>
<a href="{% url 'logout' %}">Logout</a>
{% else %}
<a href="{% url 'login' %}">Login</a>
<a href="{% url 'register' %}">Register</a>
{% endif %}

```

→ ■ Day 4 Lab

1. Add login, logout, and register URLs to `taskmanager/urls.py`
2. Create `tasks/templates/tasks/login.html` and `register.html` (both use `{{ form.as_p }}`)
3. Add `LOGIN_URL`, `LOGIN_REDIRECT_URL`, `LOGOUT_REDIRECT_URL` to `settings.py`
4. Add `@login_required` to all task views (`task_list`, `task_detail`, `create`, `edit`, `delete`)
5. Update `task_list` to filter by `owner=request.user`
6. Update `create_task` to assign `task.owner = request.user` (using `commit=False` before save)
7. Update `edit` and `delete` views to use `get_object_or_404(Task, pk=pk, owner=request.user)`
8. Update `admin.py list_display` to include 'owner' now that the field exists
9. Show the username and a logout link in `base.html` using `{% if user.is_authenticated %}`

■ Bonus Challenge

Test ownership security: create two user accounts. Log in as User A and create a task. Note the task's ID. Log out. Log in as User B. Try visiting `/tasks/edit/` directly in the browser. You should get a 404 — confirm it works and write down why it works.

Day 5 — Intro to DRF — Serializers & Function-Based API Views

What is a REST API, installing DRF, serializers, your first endpoints

■ What is a REST API and Why Do We Need One?

Our Task Manager web app works great in a browser. But what if someone wants to build a mobile app or a React frontend for the same tasks? They can't use HTML — they need raw data in a structured format.

A **REST API** (Representational State Transfer API) exposes your data as **JSON** over HTTP. Instead of returning an HTML page, your server returns something like:

```
{
  "id": 1,
  "title": "Fix login bug",
  "status": "in_progress",
  "priority": "high",
  "due_date": "2026-05-15"
}
```

Any app — mobile, desktop, another website — can read this JSON and display it however they want.

The Key Insight

We are NOT changing our Task model or our database. DRF simply gives us a NEW way to access the same data — via JSON instead of HTML. Days 1–4 serve HTML. Days 5–7 serve JSON. Same models, same database, different output.

■ REST API Conventions — HTTP Methods & Status Codes

REST APIs use HTTP methods to indicate what action to perform. Every response also includes a **status code** — a number that tells the client whether it succeeded or failed.

Method	URL	What happens	Response
GET	/api/v1/tasks/	List all my tasks	200 OK
POST	/api/v1/tasks/	Create a new task	201 Created
GET	/api/v1/tasks/1/	Get one task	200 OK
PUT	/api/v1/tasks/1/	Replace a task fully	200 OK
PATCH	/api/v1/tasks/1/	Update specific fields	200 OK
DELETE	/api/v1/tasks/1/	Delete a task	204 No Content

Common status codes: **200** = OK, **201** = Created, **204** = No Content (success, nothing to return), **400** = Bad Request (your data is wrong), **401** = Unauthorized (not logged in), **403** = Forbidden (logged in but not allowed), **404** = Not Found.

■ Installing Django REST Framework

```
pip install djangorestframework==3.15.*
pip freeze > requirements.txt # Save to requirements file
```

Add to **settings.py**:

```
INSTALLED_APPS = [
    ...
    'rest_framework', # The DRF framework itself
    'rest_framework.authtoken', # Token authentication (used in Day 6)
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
        'rest_framework.authentication.SessionAuthentication',
        # SessionAuthentication keeps the browsable API working in the browser
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
        # By default, every API endpoint requires the user to be logged in.
    ],
}
```

■ Serializers — Converting Models to JSON

A **Serializer** is DRF's equivalent of a Form — but for APIs. It does two things: converts a Django model instance into JSON data (**serialization**), and converts incoming JSON into a validated Python object (**deserialization**).

■ *Analogy: A serializer is a translator. Your Django model speaks Python objects. Your API client speaks JSON. The serializer translates in both directions: Python → JSON when sending data out, JSON → Python when receiving data in.*

```
# tasks/serializers.py (create this new file)
from rest_framework import serializers
from .models import Task

class TaskSerializer(serializers.ModelSerializer):
    # ModelSerializer auto-generates fields from the model – like ModelForm does for forms.

    owner_username = serializers.CharField(source='owner.username', read_only=True)
    # source='owner.username' means: follow the owner ForeignKey to the User model
    # and get the username field from there.
    # read_only=True means: include this in responses but never accept it as input.

    status_display = serializers.CharField(source='get_status_display', read_only=True)
    priority_display = serializers.CharField(source='get_priority_display', read_only=True)
    # These call the Django methods that return human-readable labels.
```

```

class Meta:
# class Meta here works the same as in ModelForm –
# it's where you configure the serializer itself.
model = Task
fields = [
'id', 'title', 'description',
'status', 'status_display',
'priority', 'priority_display',
'due_date', 'created_at', 'updated_at',
'owner', 'owner_username',
]
read_only_fields = ['id', 'created_at', 'updated_at', 'owner']
# read_only_fields = fields that are included in responses
# but are NEVER accepted as input from the client.

```

■■ Function-Based API Views

DRF's `@api_view` decorator turns a regular Python function into an API view. It works similarly to Django views but handles JSON automatically and provides DRF's browsable API interface.

```

# tasks/api_views.py (create this new file)
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
# Response is DRF's version of HttpResponse. It automatically serializes data to JSON.
from rest_framework import status
# status is a module with readable constants: status.HTTP_201_CREATED instead of just 201.
from .models import Task
from .serializers import TaskSerializer

@api_view(['GET', 'POST'])
# @api_view(['GET', 'POST']) means: this view accepts GET and POST requests only.
# Any other method (PUT, DELETE, etc.) will get a 405 Method Not Allowed response.
@permission_classes([IsAuthenticated])
def task_list_api(request):
if request.method == 'GET':
tasks = Task.objects.filter(owner=request.user)
serializer = TaskSerializer(tasks, many=True)
# many=True tells the serializer: 'this is a list of objects, not just one'
return Response(serializer.data)
# serializer.data is the Python dict/list representation.
# Response() converts it to JSON and returns it.

elif request.method == 'POST':
serializer = TaskSerializer(data=request.data)
# request.data is the incoming JSON parsed into a Python dict.
# (Like request.POST but works for JSON, form data, and more.)
if serializer.is_valid():
serializer.save(owner=request.user)
# Calling serializer.save() calls Task.objects.create() internally.
# We pass owner=request.user to set the owner field (it's read_only in the serializer).
return Response(serializer.data, status=status.HTTP_201_CREATED)
return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
# serializer.errors is a dict of field names and their error messages.

```

```

@api_view(['GET', 'PUT', 'DELETE'])
@permission_classes([IsAuthenticated])
def task_detail_api(request, pk):
    task = Task.objects.filter(pk=pk, owner=request.user).first()
    if not task:
        return Response({'error': 'Not found'}, status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        return Response(TaskSerializer(task).data)

    elif request.method == 'PUT':
        serializer = TaskSerializer(task, data=request.data)
        # Passing 'task' as the first argument means: update this existing task.
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    elif request.method == 'DELETE':
        task.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
        # 204 No Content = success, but nothing to send back.

```

Create **tasks/api_urls.py** (new file) and connect it:

```

# tasks/api_urls.py
from django.urls import path
from . import api_views

urlpatterns = [
    path('tasks/', api_views.task_list_api, name='api_task_list'),
    path('tasks/<int:pk>/', api_views.task_detail_api, name='api_task_detail'),
]

# taskmanager/urls.py – add this line:
path('api/v1/', include('tasks.api_urls')),

```

■ Visit <http://127.0.0.1:8000/api/v1/tasks/> in your browser — DRF shows you a beautiful browsable API interface where you can test your endpoints without any extra tools.

Day 5 Lab

1. Install DRF and add it to `INSTALLED_APPS` and `REST_FRAMEWORK` settings
2. Create `tasks/serializers.py` with `TaskSerializer`
3. Create `tasks/api_views.py` with `task_list_api` and `task_detail_api`
4. Create `tasks/api_urls.py` and connect it at `api/v1/` in the main `urls.py`
5. Test GET `/api/v1/tasks/` and POST a new task using the DRF browsable API
6. Test PUT to update a task status to 'done', then DELETE it

■ Bonus Challenge

Add query param filtering to `task_list_api`: if the request includes `?status=pending` (i.e. `request.query_params.get('status')`), filter the queryset by that status. Test it: `GET /api/v1/tasks/?status=done` should return only completed tasks.

Day 6 — APIView & Token Authentication

Upgrade to class-based API views and secure the API with tokens

■■ APIView — Class-Based API Views

Yesterday we used the `@api_view` decorator on functions. DRF also provides **APIView** — a class-based approach where each HTTP method becomes its own method inside a class: `get()`, `post()`, `put()`, `delete()`.

Instead of writing `if request.method == 'GET':` inside one big function, you split the logic into clean separate methods. This makes the code easier to read, test, and extend — especially when endpoints have complex logic.

```
# Comparing the two approaches:

# --- @api_view (function-based) ---
@api_view(['GET', 'POST'])
def task_list_api(request):
    if request.method == 'GET':
        ...
    elif request.method == 'POST':
        ...

# --- APIView (class-based) ---
class TaskListAPIView(APIView):
    def get(self, request):
        ... # Handles GET automatically – no if statement needed

    def post(self, request):
        ... # Handles POST automatically

# tasks/api_views.py – replace the FBVs with these classes
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from rest_framework.permissions import IsAuthenticated
from .models import Task
from .serializers import TaskSerializer
from .permissions import IsOwner # custom permission – see below

class TaskListAPIView(APIView):
    permission_classes = [IsAuthenticated]
    # permission_classes is a list of permission checks run before the view.
    # IsAuthenticated blocks unauthenticated users automatically.

    def get(self, request):
        tasks = Task.objects.filter(owner=request.user)
        return Response(TaskSerializer(tasks, many=True).data)
```

```

def post(self, request):
    serializer = TaskSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save(owner=request.user)
    return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

class TaskDetailAPIView(APIView):
    permission_classes = [IsAuthenticated, IsOwner]
    # Multiple permissions = ALL must pass. IsOwner is our custom permission (below).

    def get_object(self, pk):
        return Task.objects.filter(pk=pk).first()

    def get(self, request, pk):
        task = self.get_object(pk)
        if not task:
            return Response(status=status.HTTP_404_NOT_FOUND)
        self.check_object_permissions(request, task)
        # check_object_permissions runs has_object_permission() on all permission classes.
        # This is how IsOwner gets to inspect the specific task object.
        return Response(TaskSerializer(task).data)

    def put(self, request, pk):
        task = self.get_object(pk)
        if not task:
            return Response(status=status.HTTP_404_NOT_FOUND)
        self.check_object_permissions(request, task)
        serializer = TaskSerializer(task, data=request.data)
        if serializer.is_valid():
            serializer.save()
        return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    def delete(self, request, pk):
        task = self.get_object(pk)
        if not task:
            return Response(status=status.HTTP_404_NOT_FOUND)
        self.check_object_permissions(request, task)
        task.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

```

Update `tasks/api_urls.py` to use the class-based views:

```

from .api_views import TaskListAPIView, TaskDetailAPIView
urlpatterns = [
    path('tasks/', TaskListAPIView.as_view(), name='api_task_list'),
    path('tasks/<int:pk>', TaskDetailAPIView.as_view(), name='api_task_detail'),
]
# Notice as_view() again – same reason as Django's LoginView.
# DRF's URL patterns expect a callable function.
# as_view() converts the APIView class into that callable.

```

■■ Custom Permission — IsOwner

DRF permissions have two levels. **has_permission** runs first — it checks whether the user can access this view at all (e.g., are they logged in?). **has_object_permission** runs second — it checks whether they can access THIS specific object (e.g., do they own this task?).

```
# tasks/permissions.py (create this new file)
from rest_framework.permissions import BasePermission

class IsOwner(BasePermission):
    message = 'You do not have permission to access this task.'
    # 'message' is what gets returned in the API response when access is denied.

    def has_object_permission(self, request, view, obj):
        # 'obj' is the specific Task being accessed.
        # This method returns True (allow) or False (deny).
        return obj.owner == request.user
        # If the task's owner is the current user → True → allow access.
        # If not → False → DRF returns a 403 Forbidden response.
```

■ Token Authentication — How It Works

Our web app uses sessions (Django sets a cookie after login). But mobile apps and external clients can't easily use cookies. **Token Authentication** solves this.

The flow: the client sends username and password once → Django returns a **token** (a long random string) → the client stores this token and sends it in the **Authorization header** of every future request. No cookies, no sessions.

```
# Step 1 – Add the token endpoint to taskmanager/urls.py:
from rest_framework.authtoken.views import obtain_auth_token
urlpatterns += [
    path('api/v1/auth/token/', obtain_auth_token, name='api_token'),
    path('api/v1/auth/register/', RegisterAPIView.as_view(), name='api_register'),
]

# Step 2 – Run migrate to create the token table:
python manage.py migrate
```

Add the **RegisterAPIView** to **tasks/api_views.py**:

```
from django.contrib.auth.models import User
from rest_framework.authtoken.models import Token
from rest_framework.permissions import AllowAny

class RegisterAPIView(APIView):
    permission_classes = [AllowAny]
    # AllowAny overrides the global IsAuthenticated default.
    # Registration must be open to everyone – you can't require a token to get a token.

    def post(self, request):
        username = request.data.get('username')
        password = request.data.get('password')
        if not username or not password:
            return Response({'error': 'username and password required'},
                            status=status.HTTP_400_BAD_REQUEST)
        if User.objects.filter(username=username).exists():
            return Response({'error': 'Username already taken'},
```

```
status=status.HTTP_400_BAD_REQUEST)
user = User.objects.create_user(username=username, password=password)
token = Token.objects.create(user=user)
return Response({'token': token.key}, status=status.HTTP_201_CREATED)
```

How to use your token in Postman:

```
# 1. Register: POST /api/v1/auth/register/
# Body (JSON): {"username": "alice", "password": "mypassword"}
# Response: {"token": "9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b"}

# 2. Use the token in every request:
# Add a Header: Authorization: Token 9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b

# 3. Test: GET /api/v1/tasks/ (with the Authorization header)
# Should return your tasks as JSON
```

— ■ Day 6 Lab

1. Create tasks/permissions.py with the IsOwner permission class
2. Refactor task_list_api and task_detail_api to TaskListAPIView and TaskDetailAPIView
3. Add RegisterAPIView to api_views.py
4. Update api_urls.py with as_view() and the new auth endpoints
5. Run python manage.py migrate to create the token table
6. In Postman: register a user, get the token, use it to list and create tasks
7. Try accessing another user's task with your token — confirm you get 403

■ Bonus Challenge

Add PATCH support to TaskDetailAPIView. PATCH is a partial update — the client only sends the fields they want to change, not all fields. In the view: `serializer = TaskSerializer(task, data=request.data, partial=True)`. The `partial=True` argument makes all fields optional during validation.

Day 7 — ViewSets, Routers & Filtering

DRF's most powerful feature — automatic full CRUD with minimal code

■ What is a ViewSet?

A **ViewSet** is a class that combines all CRUD operations into one place. Instead of writing separate classes for the list endpoint and the detail endpoint, a ViewSet handles both. And **ModelViewSet** goes further — it provides all six actions automatically: list, create, retrieve, update, partial_update, and destroy.

How Much Code Does This Save?

Day 5 with `@api_view`: ~60 lines for both endpoints. Day 6 with `APIView`: ~50 lines. Day 7 with `ModelViewSet`: ~20 lines. Same functionality. This is why ViewSets exist — they eliminate the repetitive boilerplate.

```
# tasks/api_views.py — replace TaskListAPIView and TaskDetailAPIView with this
from rest_framework import viewsets, filters
from rest_framework.permissions import IsAuthenticated
from .models import Task
from .serializers import TaskSerializer
from .permissions import IsOwner

class TaskViewSet(viewsets.ModelViewSet):
    # ModelViewSet automatically provides these actions:
    # list() → GET /api/v1/tasks/
    # create() → POST /api/v1/tasks/
    # retrieve() → GET /api/v1/tasks/{pk}/
    # update() → PUT /api/v1/tasks/{pk}/
    # partial_update() → PATCH /api/v1/tasks/{pk}/
    # destroy() → DELETE /api/v1/tasks/{pk}/

    serializer_class = TaskSerializer
    permission_classes = [IsAuthenticated, IsOwner]

    def get_queryset(self):
        # get_queryset() is called automatically every time the ViewSet needs data.
        # We override it to filter tasks by the current user and apply optional filters.
        qs = Task.objects.filter(owner=self.request.user)

        # Read ?status= from the URL query parameters
        status_f = self.request.query_params.get('status')
        if status_f:
            qs = qs.filter(status=status_f)

        priority_f = self.request.query_params.get('priority')
        if priority_f:
            qs = qs.filter(priority=priority_f)

        return qs
```

```
def perform_create(self, serializer):
    # perform_create() is called when a POST request saves a new task.
    # We override it to assign the logged-in user as owner automatically.
    serializer.save(owner=self.request.user)
```

■■ Router — Automatic URL Generation

Writing URL patterns manually for every ViewSet action would be tedious. A **Router** reads your ViewSet and generates all URL patterns automatically. You register the ViewSet once and the router creates all routes.

```
# tasks/api_urls.py – replace everything with the router
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from rest_framework.auth_token.views import obtain_auth_token
from .api_views import TaskViewSet, RegisterAPIView

router = DefaultRouter()
router.register(r'tasks', TaskViewSet, basename='task')
# router.register() auto-creates these URL patterns:
# GET POST /api/v1/tasks/
# GET PUT PATCH DELETE /api/v1/tasks/{pk}/

urlpatterns = [
    path('', include(router.urls)),
    path('auth/token/', obtain_auth_token, name='api_token'),
    path('auth/register/', RegisterAPIView.as_view(), name='api_register'),
]
```

■ SearchFilter & OrderingFilter

filter_backends is a ViewSet setting that plugs in automatic filtering behavior. You add it to your ViewSet class and DRF handles the rest.

SearchFilter enables a `?search=` query parameter. Django performs a case-insensitive LIKE search across all fields listed in `search_fields`. For example, `?search=bug` returns tasks where the title or description contains 'bug'.

OrderingFilter enables a `?ordering=` query parameter. Clients can sort results by any field listed in `ordering_fields`. A `-` prefix means descending: `?ordering=-created_at` = newest first.

```
class TaskViewSet(viewsets.ModelViewSet):
    serializer_class = TaskSerializer
    permission_classes = [IsAuthenticated, IsOwner]

    # Add these three lines to enable filtering and ordering:
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ['title', 'description']
    ordering_fields = ['created_at', 'due_date', 'priority']
    ordering = ['-created_at'] # default ordering if no ?ordering= param

    def get_queryset(self): ...
    def perform_create(self, serializer): ...
```

```
# Now these URLs all work automatically:
# GET /api/v1/tasks/?status=pending → only pending tasks
# GET /api/v1/tasks/?search=bug → tasks with 'bug' in title or description
# GET /api/v1/tasks/?ordering=due_date → sorted by due date ascending
# GET /api/v1/tasks/?ordering=-created_at → newest first
# GET /api/v1/tasks/?status=done&ordering=-created_at → combine filters
```

■ Pagination

When you have hundreds of tasks, returning all of them in one response is slow. **Pagination** splits results into pages. Add this to settings.py inside REST_FRAMEWORK:

```
REST_FRAMEWORK = {
    ...
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 10,
    # Now GET /api/v1/tasks/ returns at most 10 tasks.
    # The response includes:
    # {
    #   "count": 42, ← total number of tasks
    #   "next": "/api/v1/tasks/?page=2", ← URL for next page (null if last page)
    #   "previous": null, ← URL for previous page (null if first page)
    #   "results": [...] ← the 10 tasks for this page
    # }
}
```

■ Custom Actions with @action

A ViewSet's default actions are list, create, retrieve, update, partial_update, and destroy. But what if you need a custom endpoint that doesn't fit those? The **@action** decorator lets you add custom routes to a ViewSet.

detail=False means the action applies to the collection (like the list endpoint — no pk in the URL).

detail=True means it applies to a single object (like the detail endpoint — pk is in the URL).

```
from rest_framework.decorators import action

class TaskViewSet(viewsets.ModelViewSet):
    ...

    @action(detail=False, methods=['get'])
    def summary(self, request):
        # This creates a new endpoint at: GET /api/v1/tasks/summary/
        # 'detail=False' means it's a collection-level action (no pk in URL).
        # 'methods=['get']' means only GET requests are accepted.

        qs = self.get_queryset() # Reuse the existing filtered queryset
        data = {
            'total': qs.count(),
            'pending': qs.filter(status='pending').count(),
            'in_progress': qs.filter(status='in_progress').count(),
            'done': qs.filter(status='done').count(),
        }
        return Response(data)
    # Response: {"total": 12, "pending": 5, "in_progress": 3, "done": 4}
```

— ■ Day 7 Lab

1. Replace TaskListAPIView and TaskDetailAPIView with TaskViewSet
2. Replace the manual URL patterns with DefaultRouter
3. Add filter_backends, search_fields, and ordering_fields to the ViewSet
4. Add pagination to REST_FRAMEWORK settings
5. Test all auto-generated routes: list, create, retrieve, update, partial_update, destroy
6. Test ?status=pending, ?search=, and ?ordering= in Postman
7. Read the @action explanation above until you understand it

■ Bonus Challenge

Implement the summary action yourself. Add the @action method to TaskViewSet and verify it works at GET /api/v1/tasks/summary/. Then add a second action: @action(detail=True, methods=['post']) called mark_done that sets task.status = 'done' and task.save() for a specific task. It should be accessible at POST /api/v1/tasks/{pk}/mark_done/.